

Make Your Own CI/CD Domain for IaaS Based Cloud Deployments

This tutorial shows how to create a simple CI domain, which can be used to communicate with virtual instances as well as to test and automate the deployment of your software in IaaS based cloud services.



Continuous integration and continuous development or CI/CD is one of the best modern software practices for DevOps teams deploying multiple applications in different types of servers. In IaaS based cloud software services, developers are required to configure their own servers and everything else, except the hardware components. Imagine having to regularly check for securities, operations, and development updates. This tutorial will show you how to write your own simple CI/CD bash script to automate all these tasks.

Infrastructure as a Service

(IaaS): IaaS is a cloud service model offering access to computing resources

like servers, storage, and networks. The IaaS provider hosts and manages this infrastructure. Unlike PaaS or SaaS, customers use the internet to access the hardware and resources.

Continuous integration (CI):

This is the continuous automation of integration of code changes or frequent updates at mentioned time intervals. This facilitates effective processes for getting products to market faster than ever before by ensuring an ongoing flow and bug fixes without the constant need for manual help.

Continuous development (CD):

Continuous development comprises the automated tests, builds, and packages updated in software to

ensure it is kept up-to-date and evolves from time to time. Every change that passes the automated tests is automatically placed in production so that the developers are not constrained by regulatory or other requirements.

Shell script: Shell script is the interface used by developers to communicate directly with the operating system. In local Linux based systems, one uses the `.sh` type file to create shell scripts to communicate with local machines. In platforms like Azure Cloud, Google Cloud, and AWS cloud services, this can be implemented to communicate with their virtual instances.